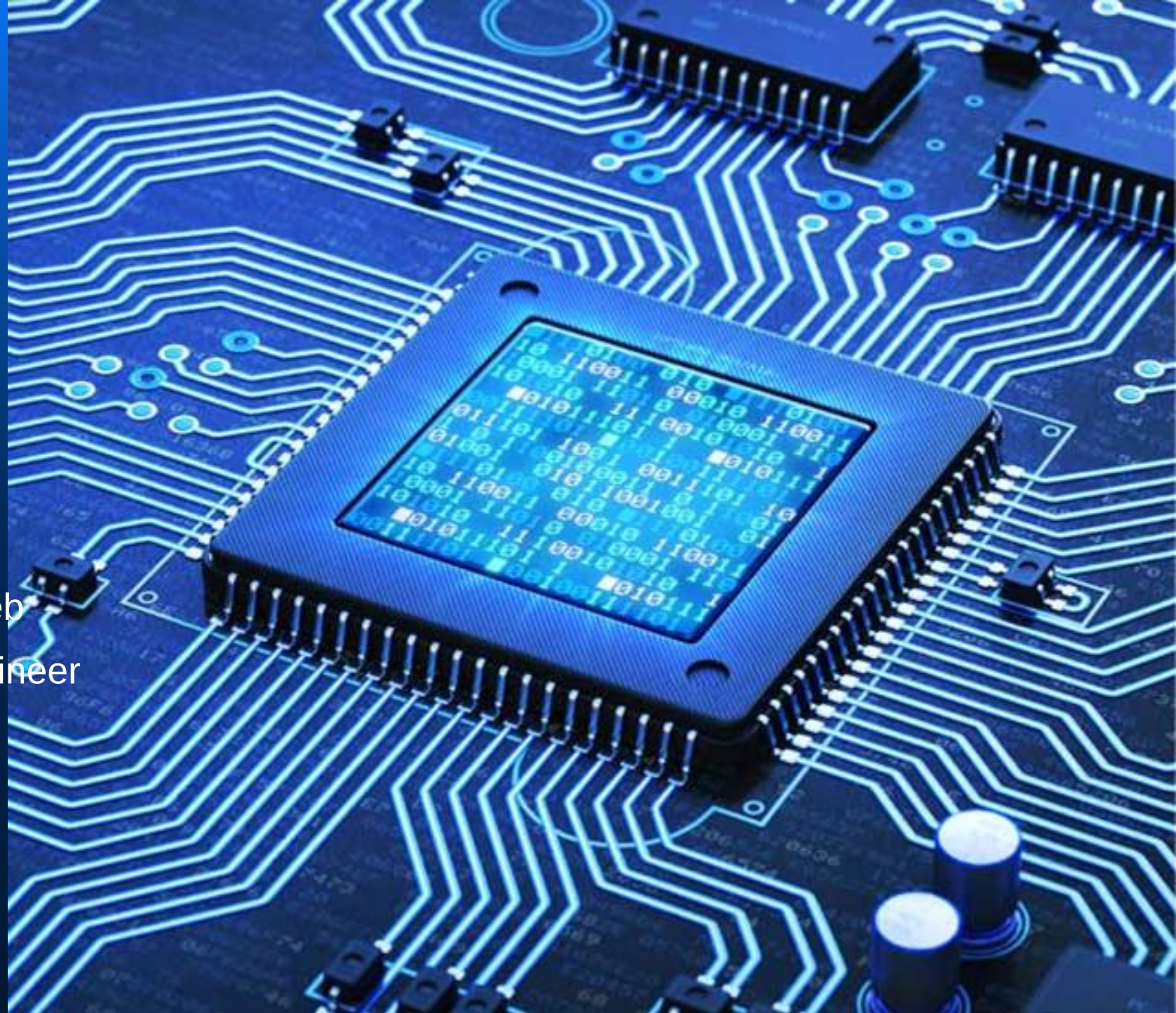
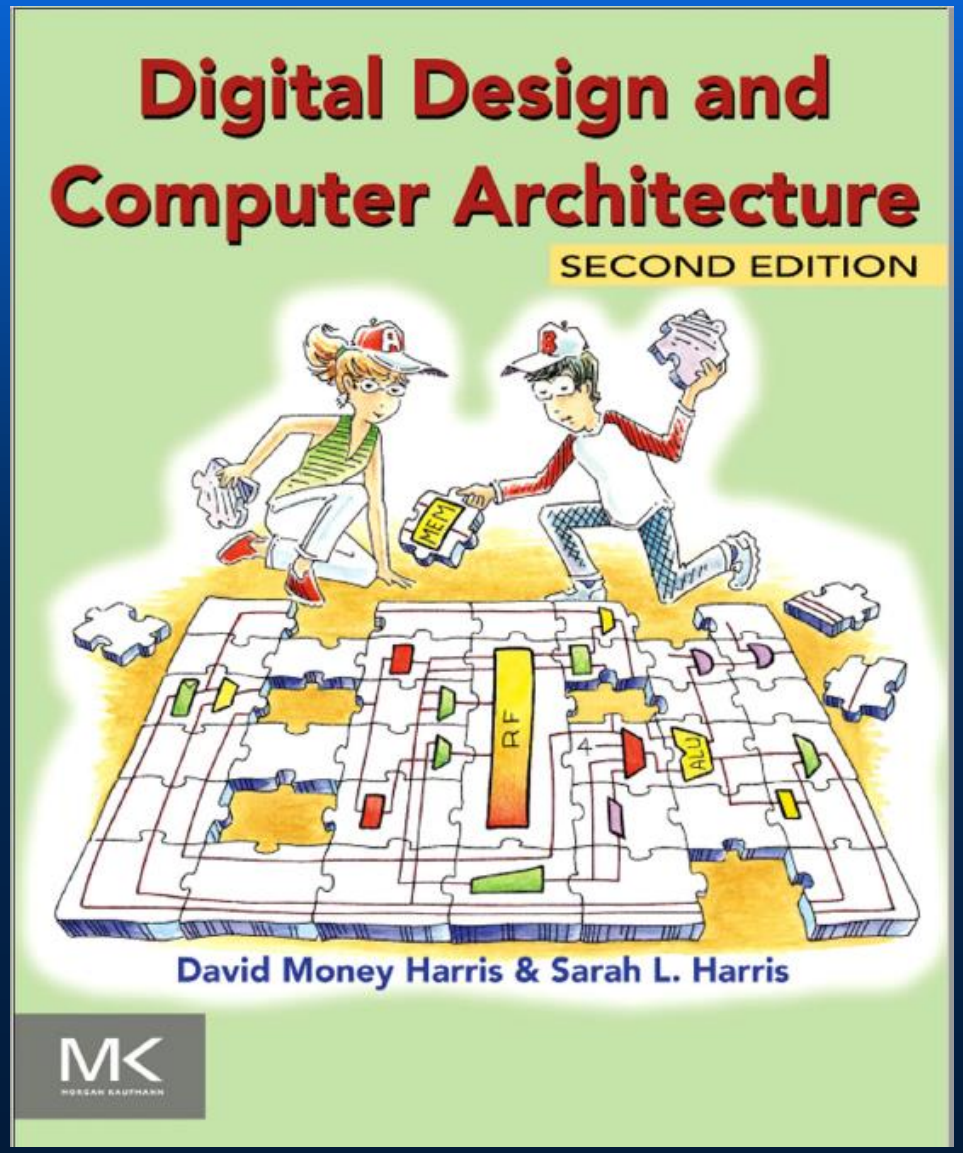
Computer Architecture Session 4

By: Eng. Abdulkareem Abotaleb

Senior HW Digital Design Engineer





This presentation is prepared from:

Digital Design and Computer Architecture, 2nd Edition.
David Harris and Sarah L. Harris.

Agenda

Chapter 7: Microarchitecture

- # 5. Pipelined Processor
- # 6. Exceptions

5. Pipelined Processor

life.augmented

5. Pipelined Processor

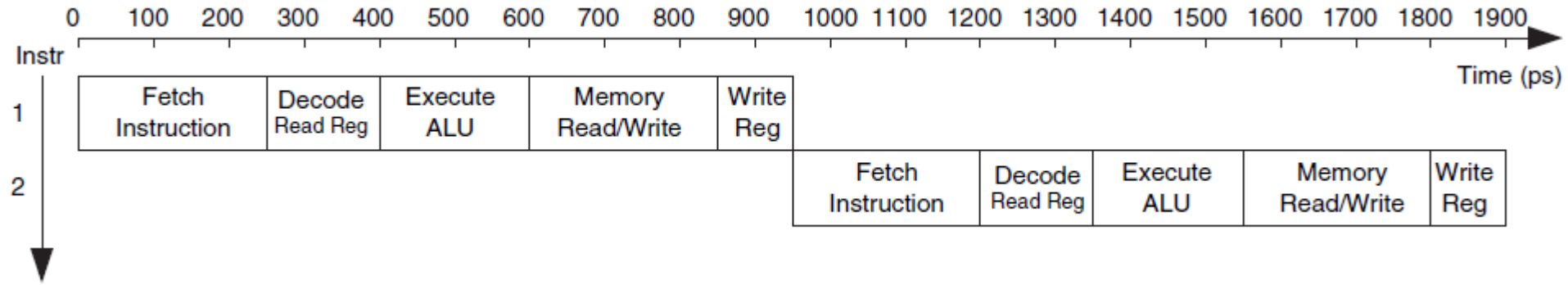
- Pipelining is a powerful way to improve the throughput of a digital system.
- We design a pipelined processor by dividing the single-cycle processor into 5 stages, to be executed simultaneously, one in each stage.
- Because each stage has only 1/5 the logic, the clock **frequency** is almost 5 times faster.
- Hence, the **latency** is ideally unchanged, but the **throughput** is ideally 5 times better.
- Pipelining introduces some overhead, so the throughput will not be quite as high as we might ideally desire, but pipelining gives such great advantage.

5. Pipelined Processor

- Using the memory, register file and ALU are the biggest delays.
- We choose 5 pipeline stages (Fetch, Decode, Execute, Memory, and Writeback) so that each stage involves exactly one of these slow steps.
- They are like the five steps that the multicycle processor used to perform lw.
- Fetch stage: the processor reads the instruction from **instruction memory**.
- Decode stage: the processor reads the source operands from the **register file** and encodes the instruction to produce the control signals.
- Execute stage: the processor performs a computation with the **ALU**.
- Memory stage: the processor reads or writes **data memory**.
- Writeback stage: the processor writes the result to the **register file**, when applicable.

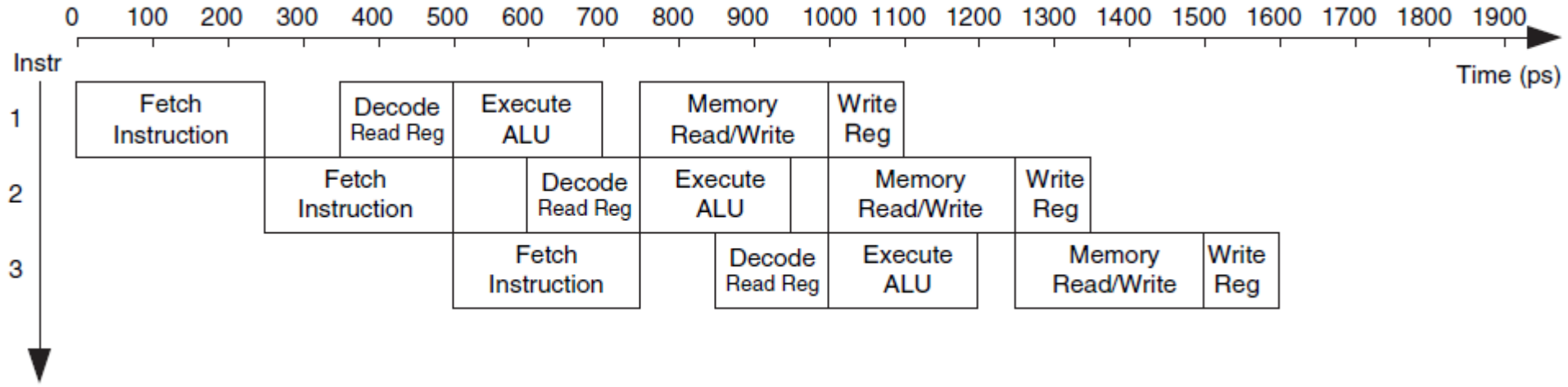
5. Pipelined Processor

- In the single-cycle processor, the first instruction is read from memory at time 0; next the operands are read from the register file.
- Then the ALU executes the necessary computation. Finally, the data memory may be accessed, and the result is written back to the register file by 950 ps.
- The second instruction begins when the first completes.
- Example: Single-cycle processor has a **latency** of 250 + 150 + 200 + 250 + 100 = **950 ps** and a **throughput** of 1 instruction per 950 ps (**1.05 billion inst per sec**).



5. Pipelined Processor

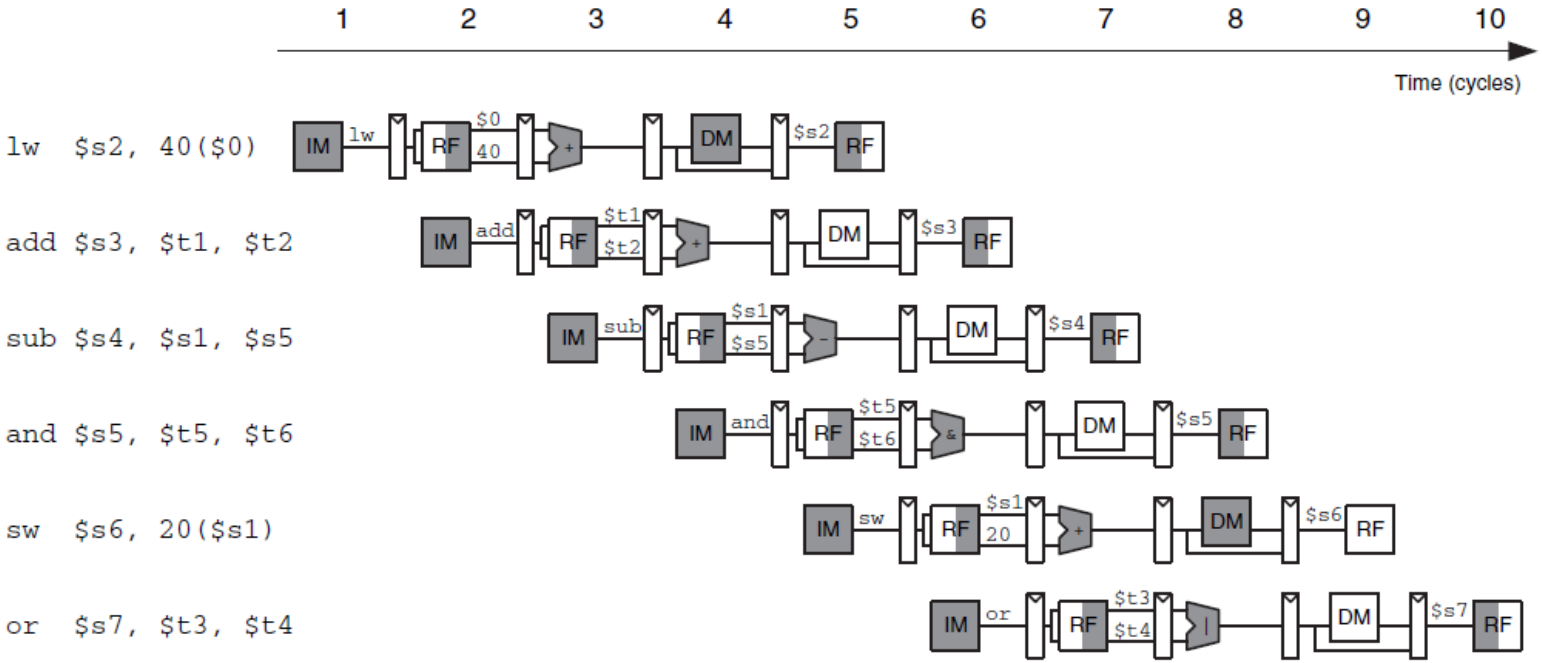
- In the pipelined processor, the length of a pipeline stage is set at 250 ps **by the slowest stage**, the memory access (in the Fetch or Memory stage).
- At time 0, the first instruction is fetched from memory.
  At 250 ps, the first instruction enters the Decode stage, while the second is fetched.
  And so on, until all the instructions complete.
- The **latency** is 5 × 250 = **1250 ps**. The **throughput** is 1 inst/250 ps (**4 billion inst/sec**).
- Because the stages are not equally balanced, the latency is slightly longer.
- Similarly, the throughput is not quite five times. Nevertheless, the throughput enhanced.



5. Pipelined Processor

Example:

- The data memory is used by **lw** in cycle 4 and by **sw** in cycle 8.
- The instruction memory and ALU are used in every cycle.
- The register file is written by every instruction except **sw**.



5. Pipelined Processor

- In the pipelined processor, the register file is written in the first part of a cycle and read in the second part. This way, data can be written and read back *within a single cycle*.
- A central challenge in pipelined systems is handling hazards that occur when the results of one instruction are needed by a subsequent instruction before the former instruction has completed.
- For example, if the add used $s2 rather than $t2, a hazard would occur because the $s2 register has not been written by the **lw** by the time it is read by the add.
- We will explore **forwarding**, **stalls**, and **flushes** as methods to resolve hazards.
- Then, we will analyze the *performance* considering *overhead* and the impact of *hazards*.

5. Pipelined Processor

5.1 Pipelined Datapath

5.2 Pipelined Control

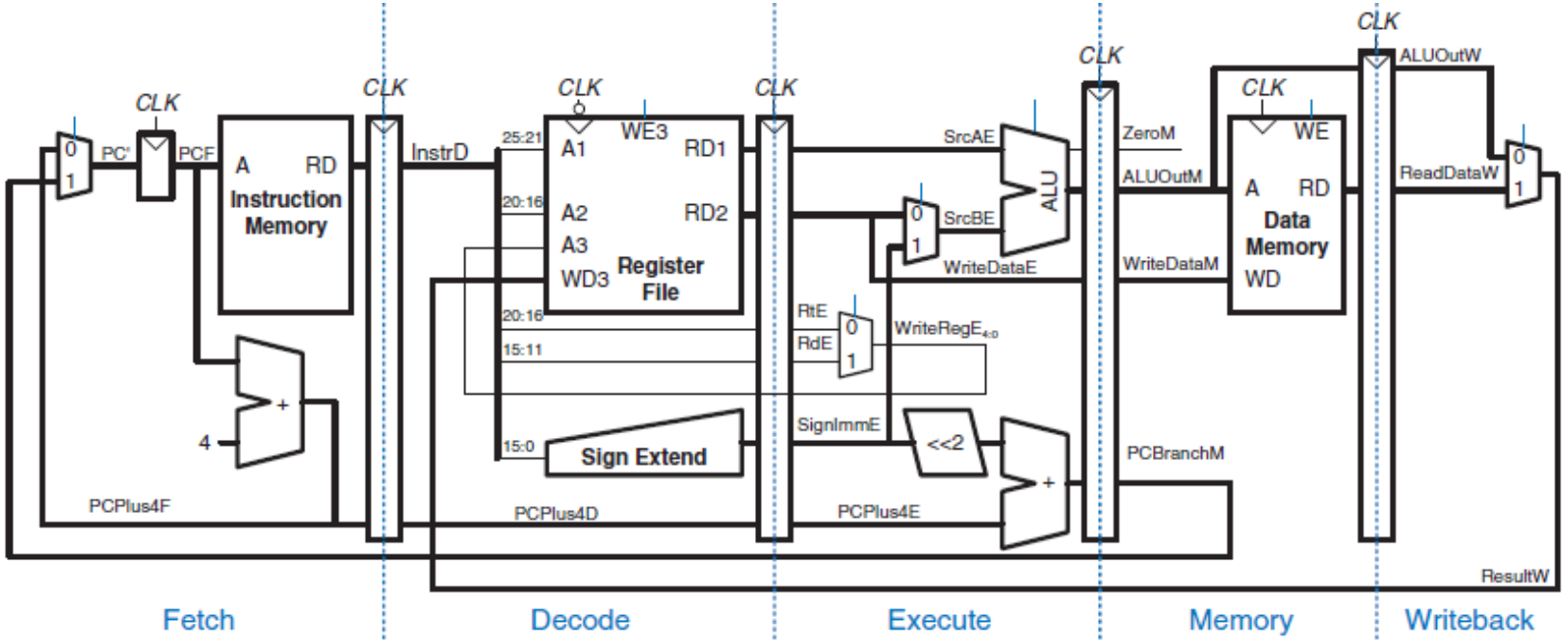
5.3 Hazards

5.4 More Instructions

5.5 Performance Analysis

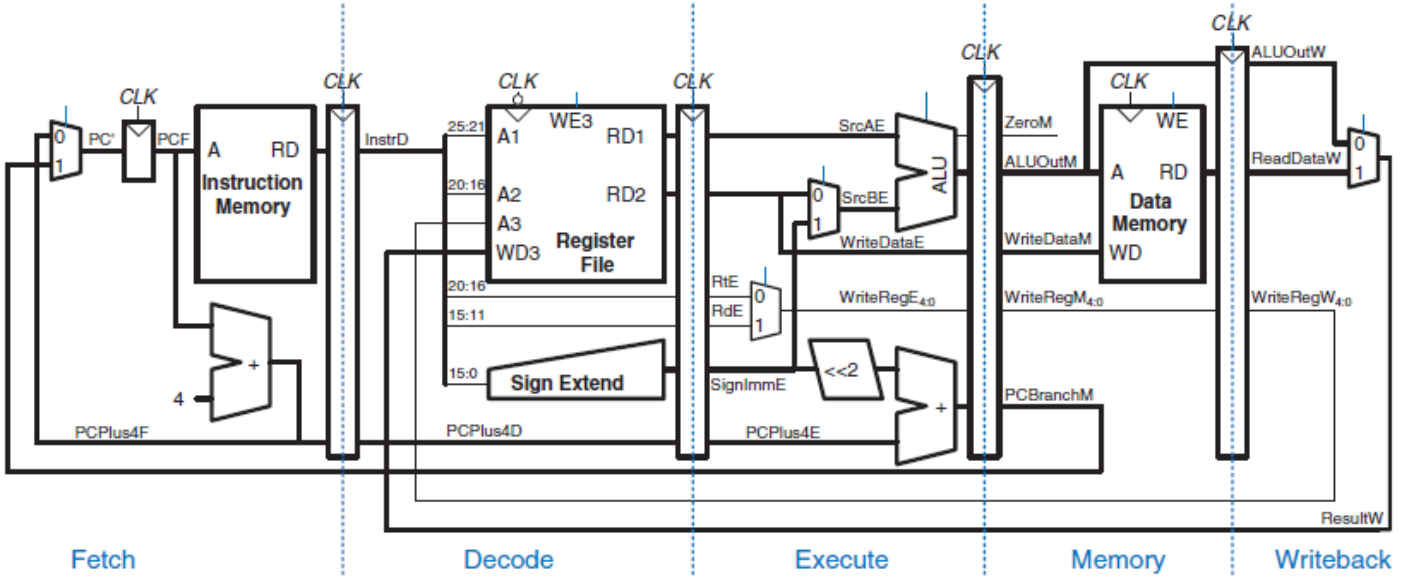
5.1 Pipelined Datapath

- The pipelined datapath is formed by dividing the single-cycle datapath into five stages separated by inserted four pipeline registers.
- The register file is read in the Decode stage and written in the Writeback stage. This feedback will lead to pipeline hazards.
- The register file writes on the falling edge of CLK, when WD3 is stable.



5.1 Pipelined Datapath

- One of the critical issues in pipelining is that all signals associated with a particular instruction should be synched through the pipeline.
- In the register file write logic, the data value comes from ResultW (Writeback stage), and the address comes from WriteRegE (Execute stage).
- The WriteReg signal should be pipelined through the Memory and Writeback stages.
- PC′ might be updated with a Fetch or a Memory stage signal (PCPlus4F or PCBranchM). This control hazard will be fixed later.



5. Pipelined Processor

5.1 Pipelined Datapath

5.2 Pipelined Control

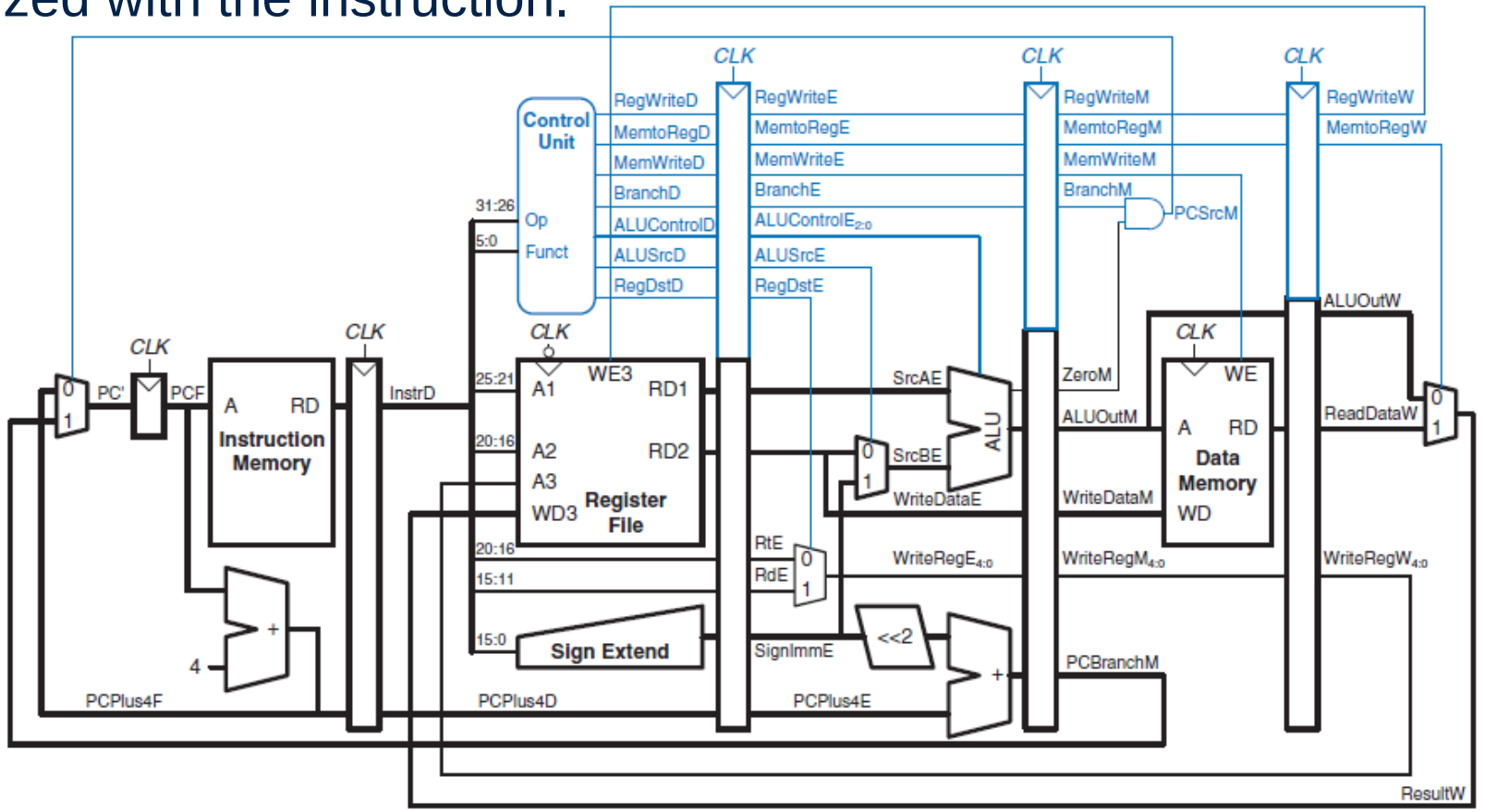
5.3 Hazards

5.4 More Instructions

5.5 Performance Analysis

5.2 Pipelined Control

- The **pipelined processor** takes the same control signals as the **single-cycle processor** and therefore uses the same control unit.
- These control signals must be pipelined along with the data so that they remain synchronized with the instruction.



5. Pipelined Processor

5.1 Pipelined Datapath

5.2 Pipelined Control

5.3 Hazards

5.4 More Instructions

5.5 Performance Analysis

5.3 Hazards

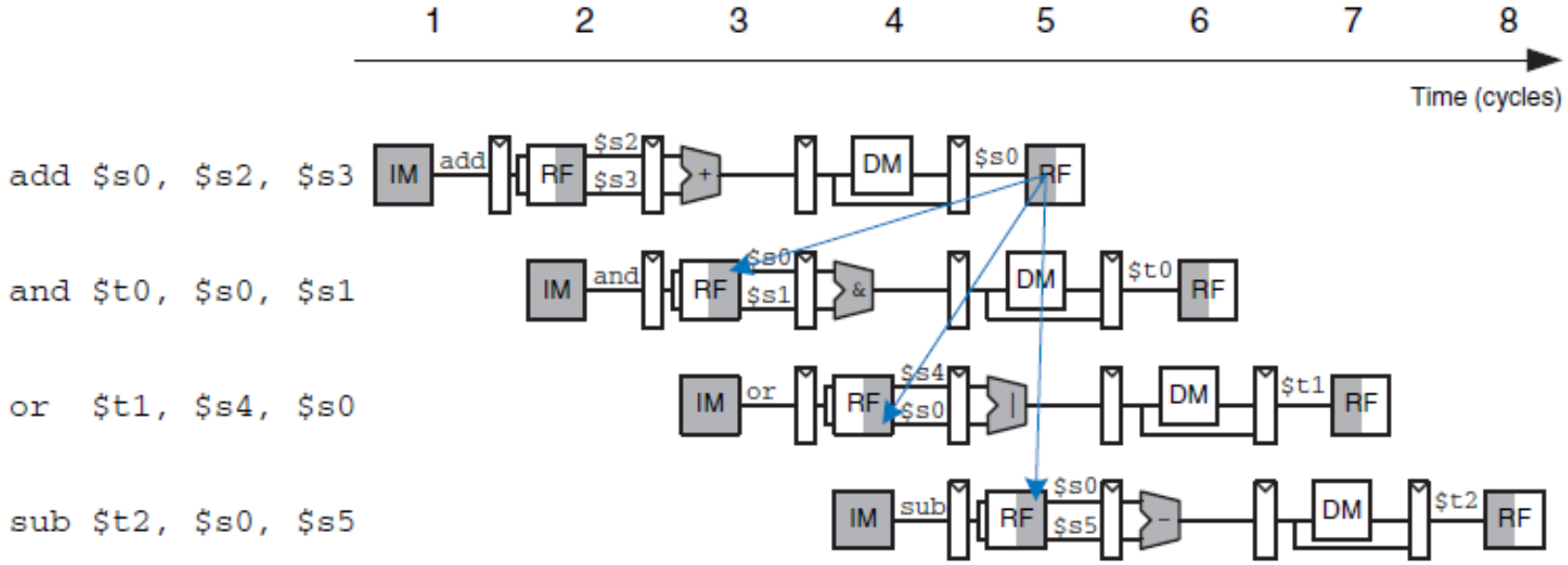
- Hazards are situations in pipelining when the next instruction cannot execute correctly in the following clock cycle.
- Hazards can be classified as **structural hazards, data hazards** or **control hazards**.
- **A structural hazard**: the hardware can't support the combination of instructions that we want to execute in the same clock cycle.
  Example: Two (or more) instructions that are already in pipeline need the same resource.
- **A data hazard** occurs when an instruction tries to read a register that has not yet been written back by a previous instruction.
- **A control hazard** occurs when the decision of what instruction to fetch next has not been made by the time the fetch takes place.
- We should enhance the pipelined processor with a hazard unit that detects hazards and handles them appropriately.

5.3 Hazards

- # Data Hazards
  - # Solving Data Hazards with Forwarding
  - # Solving Data Hazards with Stalls
- # Control Hazards

5.3 Hazards

- Data Hazard Example: The **add** instruction writes into $s0 in the first half of cycle 5. However, the **and**, **or** instructions read a wrong value on previous cycles. The **sub** instruction reads the correct value in the second half of cycle 5.
- The sum from the **add** instruction is computed by the ALU in cycle 3 and will be used by the ALU in the **and** instruction in cycle 4, so it'll be **forwarded** to the next instruction.
- In other situations, we may have to **stall** the pipeline to give time for a result to be produced before the subsequent instruction uses the result.



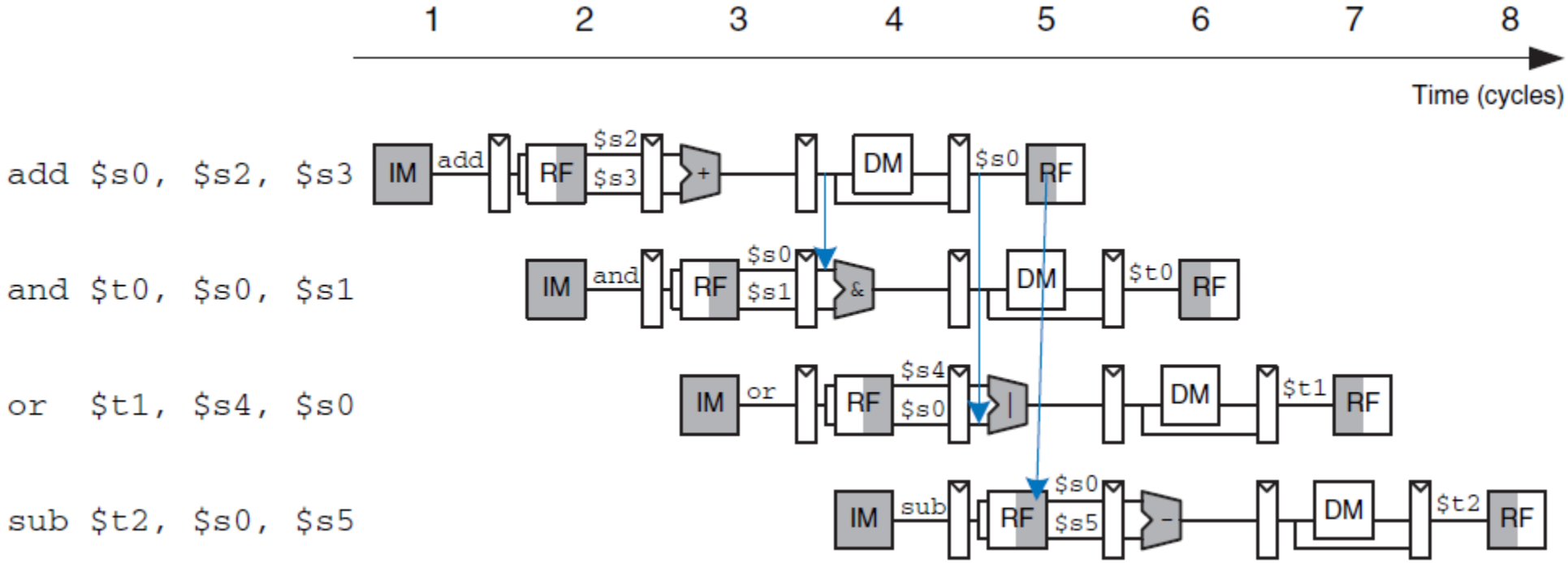
5.3 Hazards

- Data Hazards
  - Solving Data Hazards with Forwarding
  - Solving Data Hazards with Stalls
- Control Hazards

5.3 Hazards

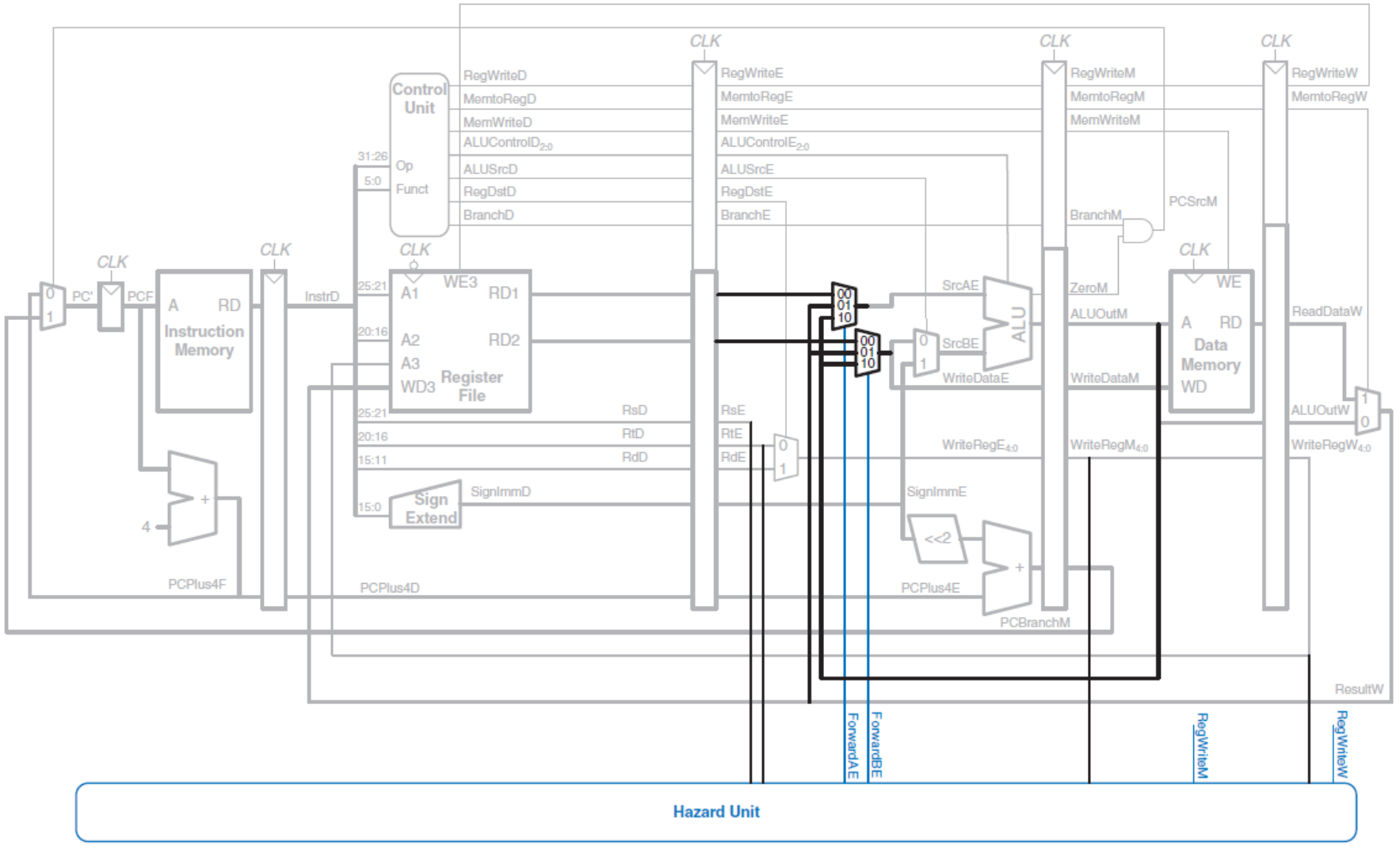
Solving Data Hazards with Forwarding:

- Some data hazards can be solved by **forwarding** (bypassing) a result from the Memory or Writeback stage to a dependent instruction in the Execute stage.
- In cycle 4, $s0 is **forwarded** from Memory stage (add inst) to Execute stage (and inst).
- In cycle 5, $s0 is **forwarded** from Writeback stage (add inst) to Execute stage (or inst).



5.3 Hazards

Solving Data Hazards with Forwarding:



5.3 Hazards

Solving Data Hazards with Forwarding:

- The pipelined processor is enhanced to support forwarding, by adding a hazard detection unit and two forwarding multiplexers.
- The **hazard detection unit** receives:
  - The two source registers from the instruction in the Execute stage
  - The destination registers from the instructions in the Memory and Writeback stages.
  - The RegWrite signals from the Memory and Writeback stages.
- The **hazard detection unit** computes control signals for the forwarding multiplexers to choose operands from the register file or from the results in Memory or Writeback stage.
- Results should forward from another stage if thst stage will write a destination register and the destination register matches the source register of the current stage.
- Note: $0 is hardwired to 0 and should never be forwarded.

5.3 Hazards

Solving Data Hazards with Forwarding:

- If both the Memory and Writeback stages contain matching destination registers, the Memory stage should have priority, as it contains the more recently executed instruction.

```
if ((rsE != 0) AND (rsE == WriteRegM) AND RegWriteM) then
        ForwardAE = 10
else if ((rsE != 0) AND (rsE == WriteRegW) AND RegWriteW) then
        ForwardAE = 01
else
        ForwardAE = 00
```

- The forwarding logic for SrcB (ForwardBE) is identical except it checks rt rather than rs.

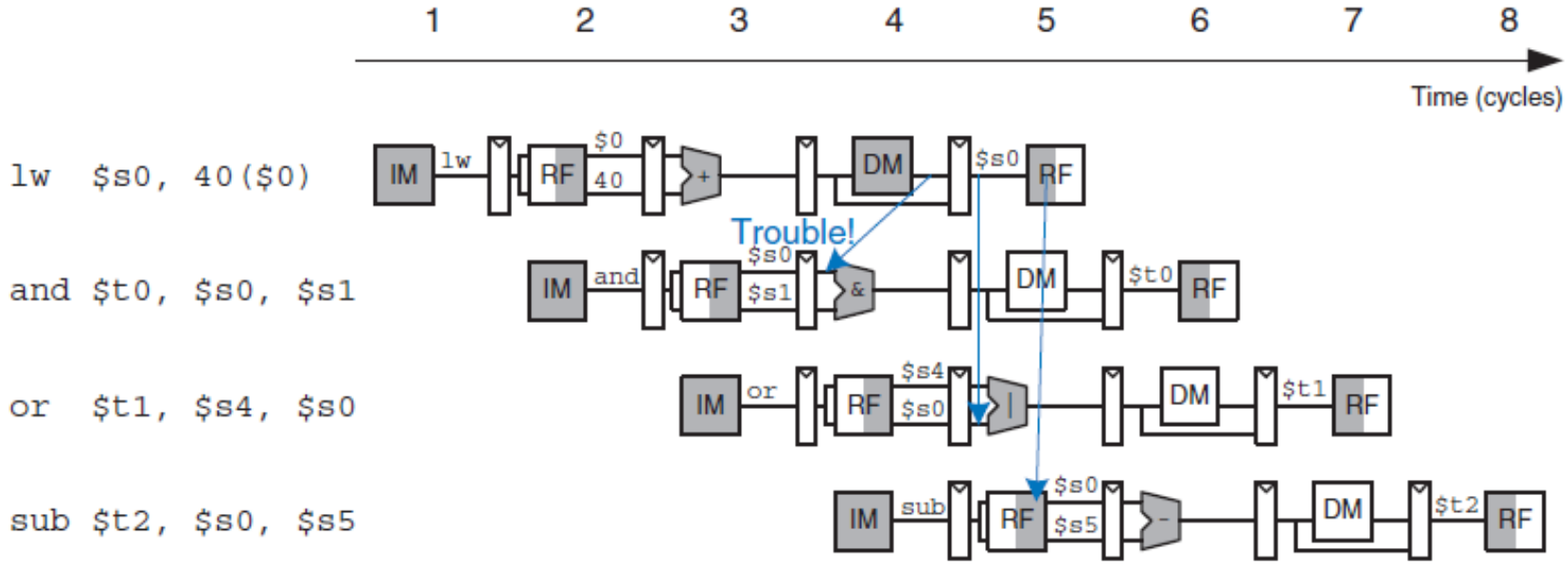
5.3 Hazards

- Data Hazards
  - Solving Data Hazards with Forwarding
  - Solving Data Hazards with Stalls
- Control Hazards

5.3 Hazards

Solving Data Hazards with Stalls:

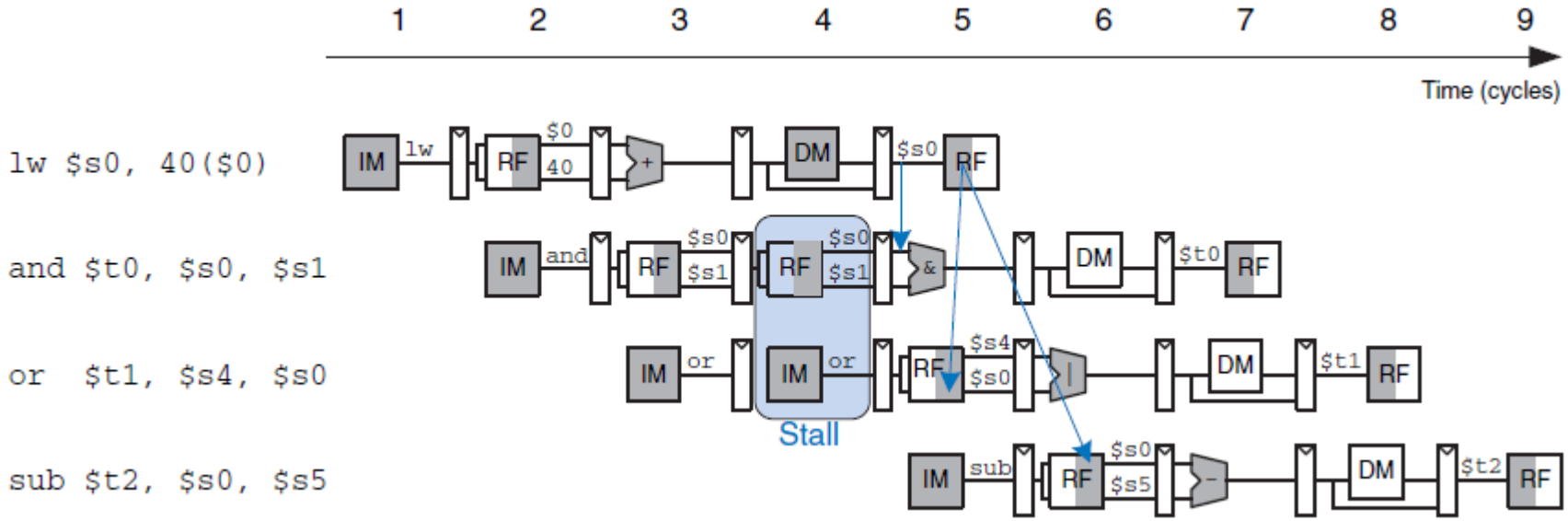
- Forwarding solves RAW data hazards when result is computed in the Execute stage.
- Unfortunately, the **lw** instruction doesn't finish reading data until the end of Memory stage.
- The **lw** instruction receives data from memory at the end of cycle 4. But the **and** instruction needs that data as a source operand at the beginning of cycle 4.



5.3 Hazards

Solving Data Hazards with Stalls:

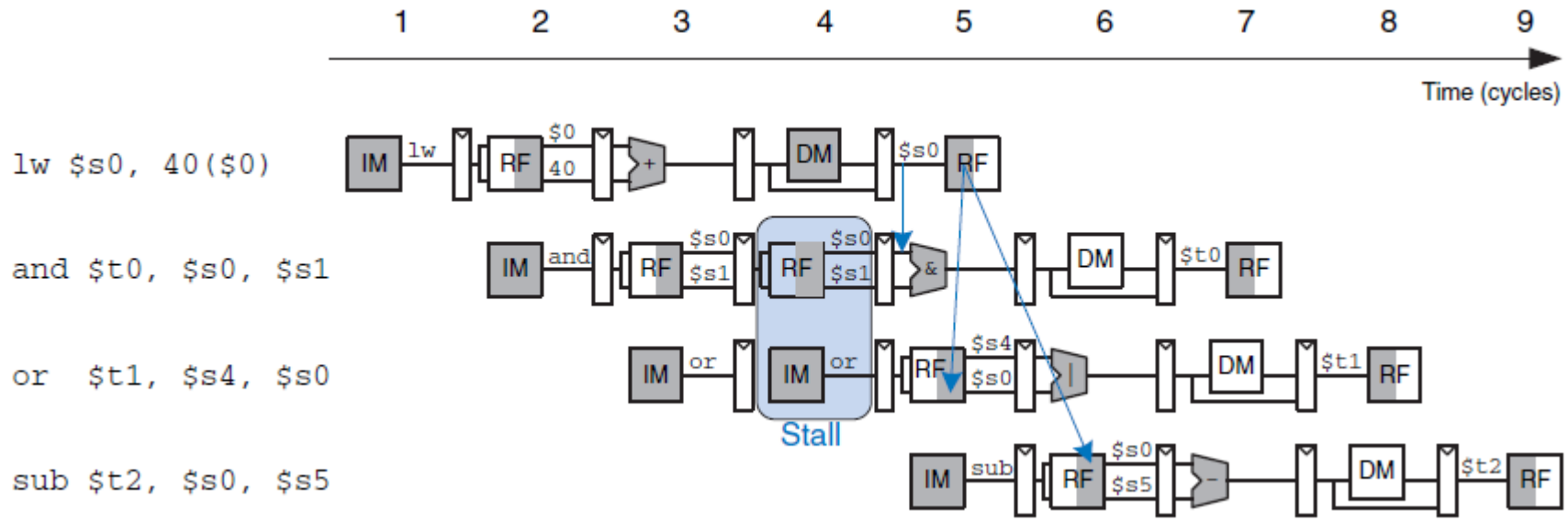
- The solution is to **stall** the pipeline, holding up operation until the data is available.
- The **and** instruction enters the Decode stage in cycle 3 and stalls there through cycle 4. The **or** instruction must remain in the Fetch stage during both cycles.
- In cycle 5, the result is forwarded from Writeback stage of **lw** to Execute stage of **and**. In cycle 5, source $s0 of **or** instruction is read directly from the register file.



5.3 Hazards

Solving Data Hazards with Stalls:

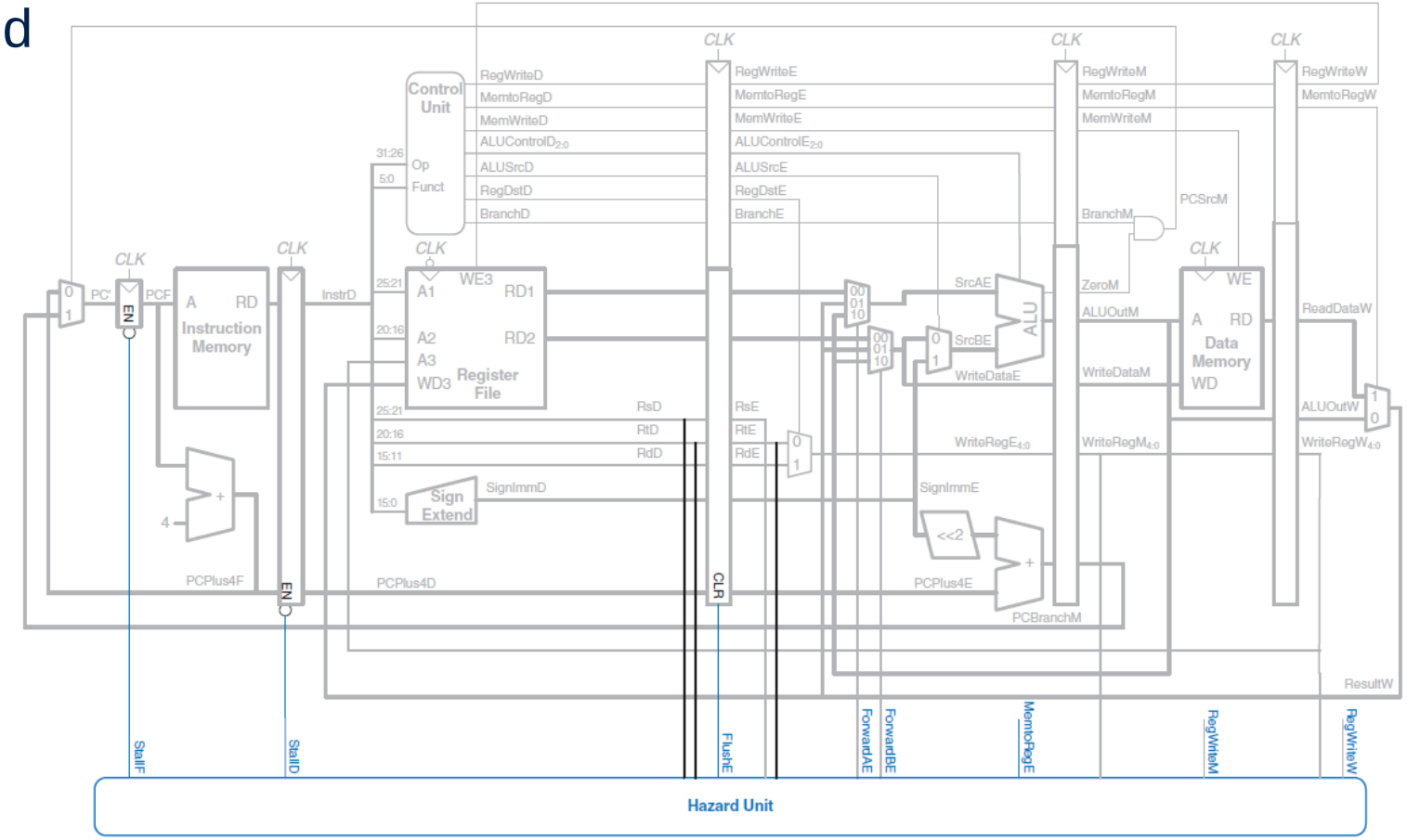
- Execute, Memory and Writeback stages are unused in cycles 4, 5 and 6 respectively.
- This propagation of unused stages via the pipeline is called a bubble like nop behavior.
- Stalling a stage is performed by disabling the pipeline register, so contents don't change.
- Stalls degrade performance, so they should only be used when necessary.



5.3 Hazards

Solving Data Hazards with Stalls:

- The pipelined processor is modified to add stalls for **lw** data dependencies.
- The hazard unit examines If the instruction in Execute stage is **lw** and its destination register (rtE) matches either source of the inst in the Decode stage (rsD or rtD), the instruction must be **stalled** in Decode stage.



5.3 Hazards

Solving Data Hazards with Stalls:

- Stalls are supported by adding
  - Enable inputs (EN) to the Fetch and Decode pipeline registers.
  - A synchronous reset/clear (CLR) input to the Execute pipeline register.
- When a **lw** stall occurs, StallD and StallF are asserted to force the Decode and Fetch stage pipeline registers to hold their old values.
  FlushE is also asserted to clear the contents of the Execute stage pipeline register.
- Remember: The MemtoReg signal is asserted for the lw instruction.

Lwstall = ((rsD== rtE) OR (rtD== rtE)) AND MemtoRegE
StallF = StallD = FlushE = lwstall

5.3 Hazards

- # Data Hazards
  - # Solving Data Hazards with Forwarding
  - # Solving Data Hazards with Stalls
- # Control Hazards

5.3 Hazards

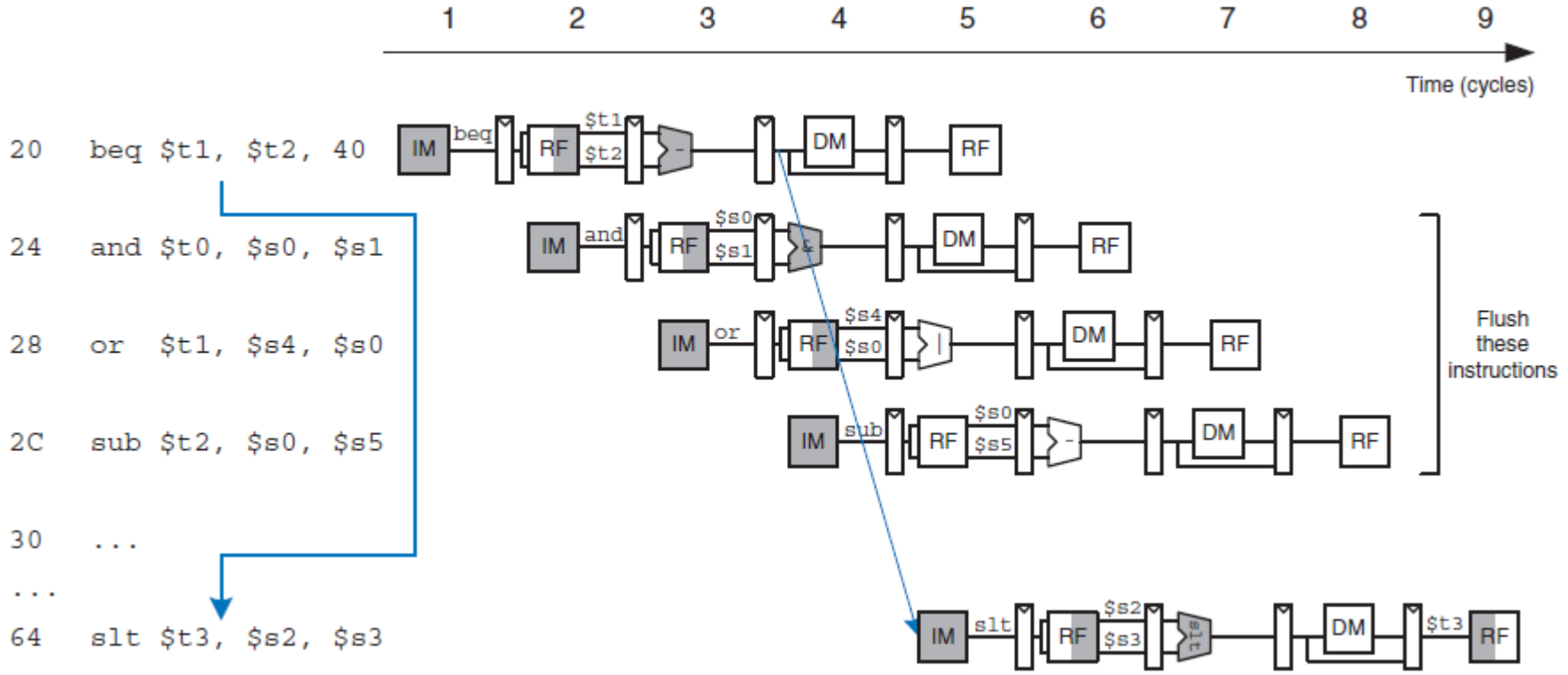
Solving Control Hazards:

- The **beq** instruction presents a control hazard: the pipelined processor does not know what instruction to fetch next, as it depends if the branch is taken or not.
- One mechanism for dealing with the control hazard is to stall the pipeline until the branch decision is made (in the Memory stage), the pipeline would have to be stalled for 3 cycles at every branch which degrades the system performance.
- An alternative is to predict whether the branch will be taken and execute instructions based on the prediction.
- if the prediction was wrong, the processor can throw out the instructions.
- Assume, we predict that branches are not taken and continue executing the program in order. If the branch is taken, the three instructions after the branch will be flushed.
- These wasted instruction cycles are called the branch misprediction penalty.

5.3 Hazards

Solving Control Hazards:

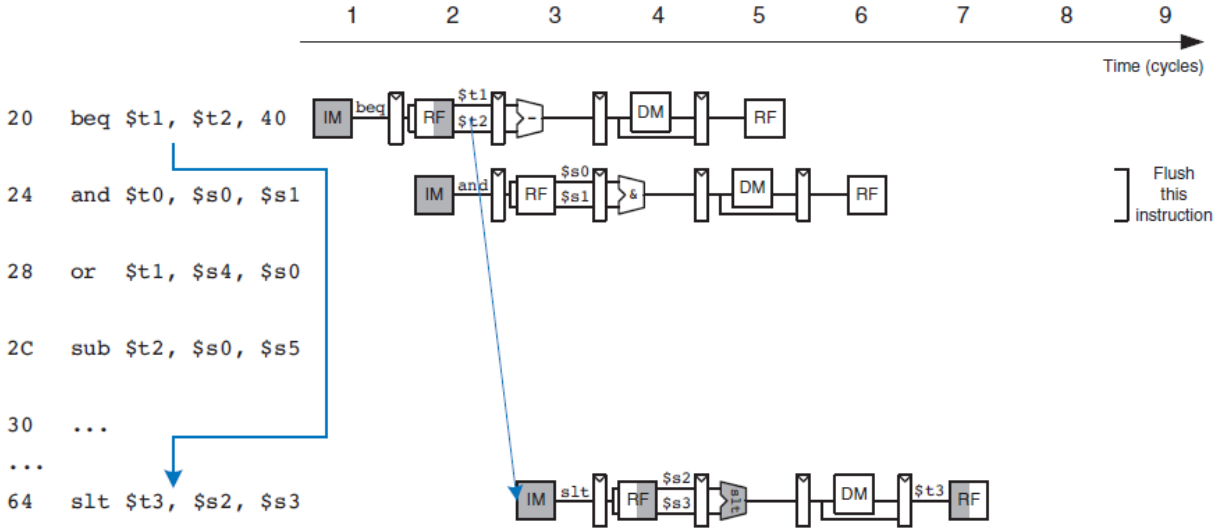
- Example: A branch from address 20 to address 64 is taken.
- The branch decision is not made until cycle 4, so the **and**, **or**, **and** sub instructions must be flushed, and the **slt** instruction is fetched in cycle 5.



5.3 Hazards

Solving Control Hazards:

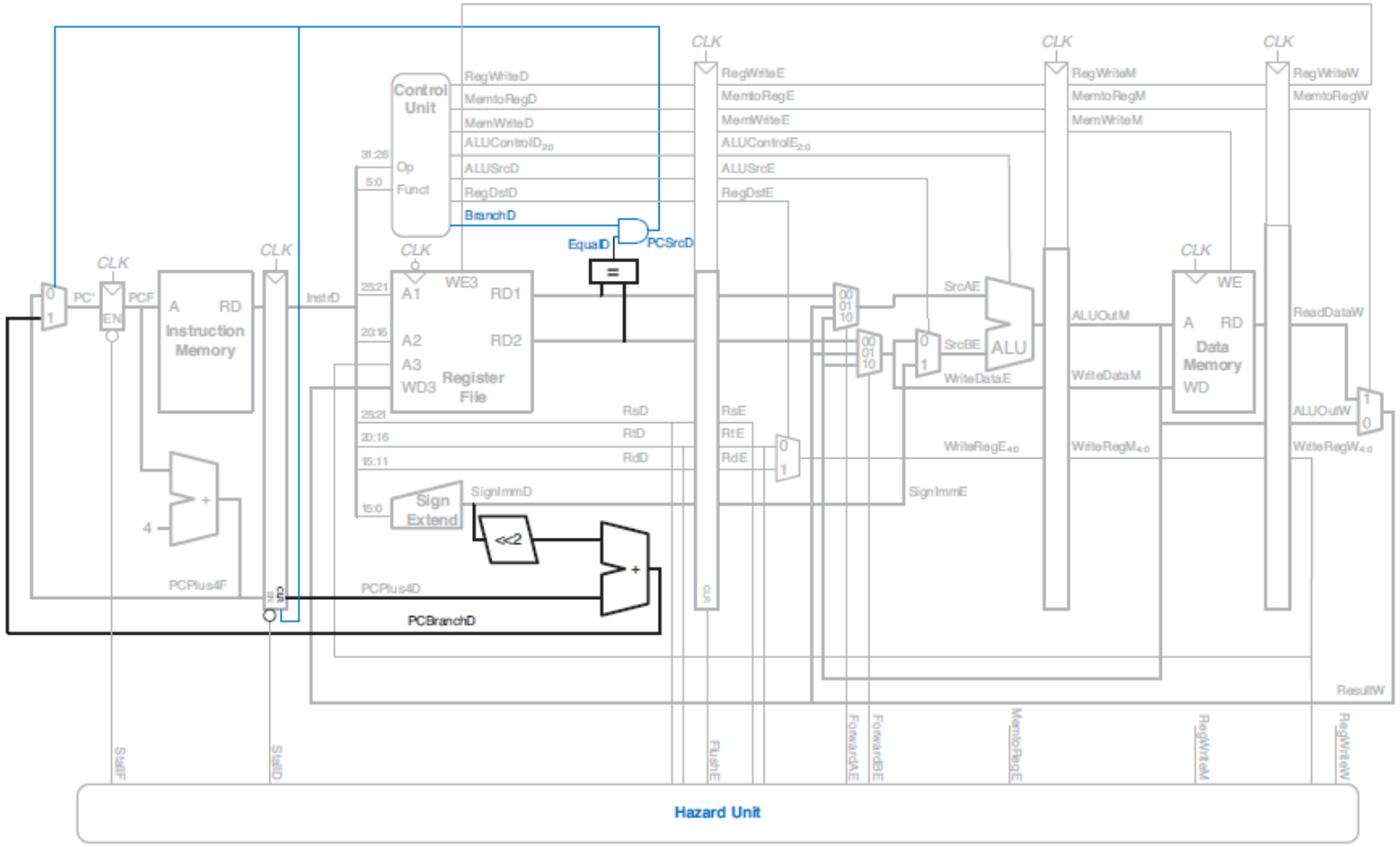
- We could reduce the branch misprediction penalty if the branch decision is made earlier.
- Making the decision simply requires comparing the values of two registers using an equality comparator instead of performing a subtraction and zero detection.
- If the comparator is fast enough, it could be moved into Decode stage, so that the operands are read and compared to determine the next PC by the end of Decode stage.
- Now the branch misprediction penalty is reduced to only one instruction.



5.3 Hazards

Solving Control Hazards:

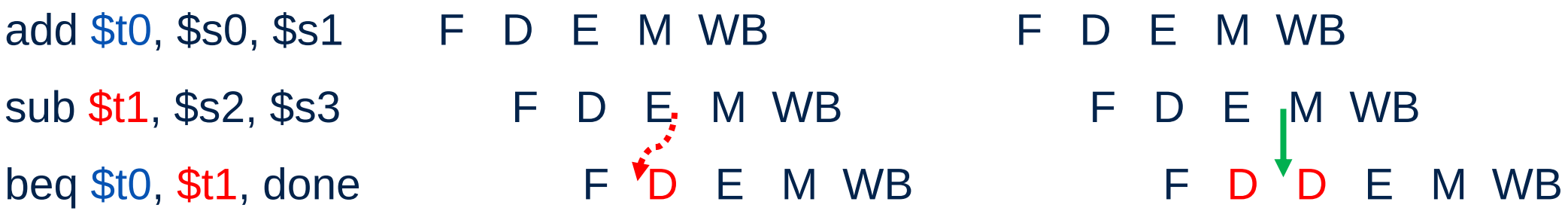
- An equality comparator and the PCSrc AND gate are moved earlier, so that PCSrc can be determined in Decode stage.
- The PCBranch adder must also be moved into Decode stage to sync the destination address.
- The synchronous clear input (CLR) is added to Decode stage pipeline register so that the wrong fetched inst can be flushed when a branch is taken.



5.3 Hazards

Solving Control Hazards:

- Unfortunately, the early branch decision hardware introduces a new RAWdata hazard.
- If one of the source operands for the branch was computed by a previous instruction and has not yet been written into the register file, the branch will read the wrong operand value from the register file.
- We can solve the data hazard by **forwarding** the correct value if it is available or by **stalling** the pipeline until the data is ready.
- Example (1):



5.3 Hazards

Solving Control Hazards:

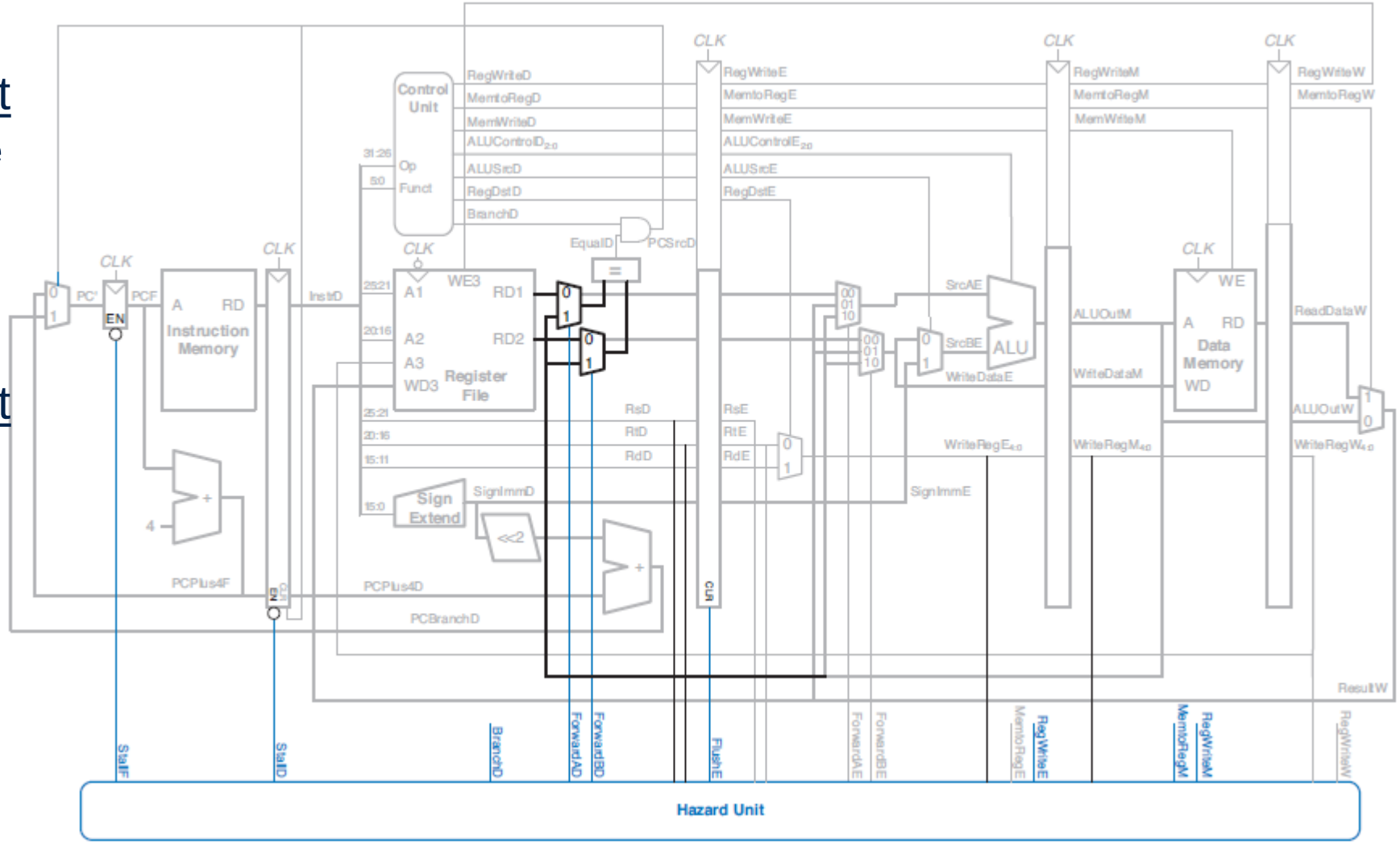
- Example (2):



5.3 Hazards

Solving Control Hazards:

- If an ALU instruction result is in Memory stage, it'll be **forwarded** to the equality comparator through two new muxs.
- If an ALU instruction result is in Execute stage or a lw instruction result is in the Memory stage, it'll be **stalled** at the Decode stage till it's ready.



5.3 Hazards

Solving Control Hazards:

- The function of the Decode stage forwarding logic is given below.

```
ForwardAD = (rsD != 0) AND (rsD == WriteRegM) AND RegWriteM
ForwardBD = (rtD != 0) AND (rtD == WriteRegM) AND RegWriteM
```

- The processor must make a branch decision in the Decode stage.
  If either of the sources of the branch depends on an ALU instruction in Execute stage or on a lw instruction in Memory stage, the processor must stall until the sources are ready.

```
branchstall = [ BranchD AND RegWriteE AND (WriteRegE == rsD OR WriteRegE == rtD) ]
              OR [ BranchD AND MemtoRegM AND (WriteRegM == rsD OR WriteRegM == rtD) ]
```

- Now the processor might stall due to either a load or a branch hazard:

```
StallF = StallD = FlushE = lwstall OR branchstall
```

5. Pipelined Processor

5.1 Pipelined Datapath

5.2 Pipelined Control

5.3 Hazards

5.4 More Instructions

5.5 Performance Analysis

5.4 More Instructions

- Supporting new instructions in the pipelined processor is like in the single-cycle processor.
- However, new instructions may introduce hazards that must be detected and solved.
- Supporting **addi** and **j** instructions in the pipelined processor requires enhancing the controller and adding a jump multiplexer to the datapath after the branch multiplexer.
- The jump takes place in the Decode stage, like a branch, so the subsequent instruction in the Fetch stage must be flushed.

5. Pipelined Processor

5.1 Pipelined Datapath

5.2 Pipelined Control

5.3 Hazards

5.4 More Instructions

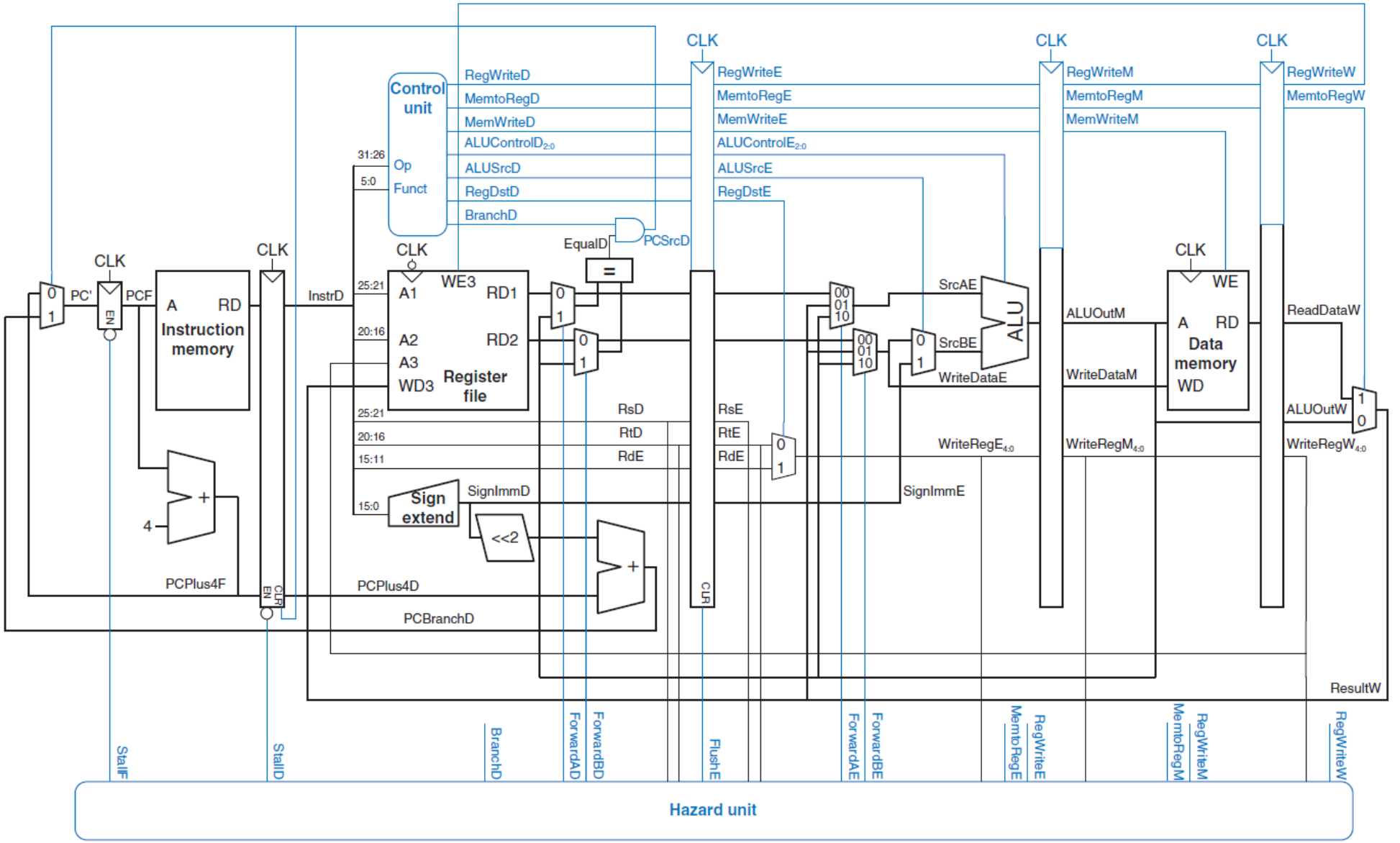
5.5 Performance Analysis

5.5 Performance Analysis

- The pipelined processor ideally has a CPI of 1, as an instruction is issued every cycle.
- However, a stall or a flush wastes a cycle, so the CPI is slightly higher and depends on the specific program being executed.
- We can determine the cycle time by considering the critical path in each of the five pipeline stages.
- Recall that the register file is written in the first half of the Writeback cycle and read in the second half of the Decode cycle. Therefore, the cycle time of the Decode and Writeback stages is twice the time necessary to do the half-cycle of work.

$$T_c = \max \left(\begin{array}{c} t_{pcq} + t_{mem} + t_{setup} \\ 2\,(t_{RFread} + t_{mux} + t_{eq} + t_{AND} + t_{mux} + t_{setup}) \\ t_{pcq} + t_{mux} + t_{mux} + t_{ALU} + t_{setup} \\ t_{pcq} + t_{memwrite} + t_{setup} \\ 2\,(t_{pcq} + t_{mux} + t_{RFread}) \end{array} \right) \begin{array}{l} \text{Fetch} \\ \text{Decode} \\ \text{Execute} \\ \text{Memory} \\ \text{Writeback} \end{array}$$

5.5 Performance Analysis



6. Exceptions

6. Exceptions

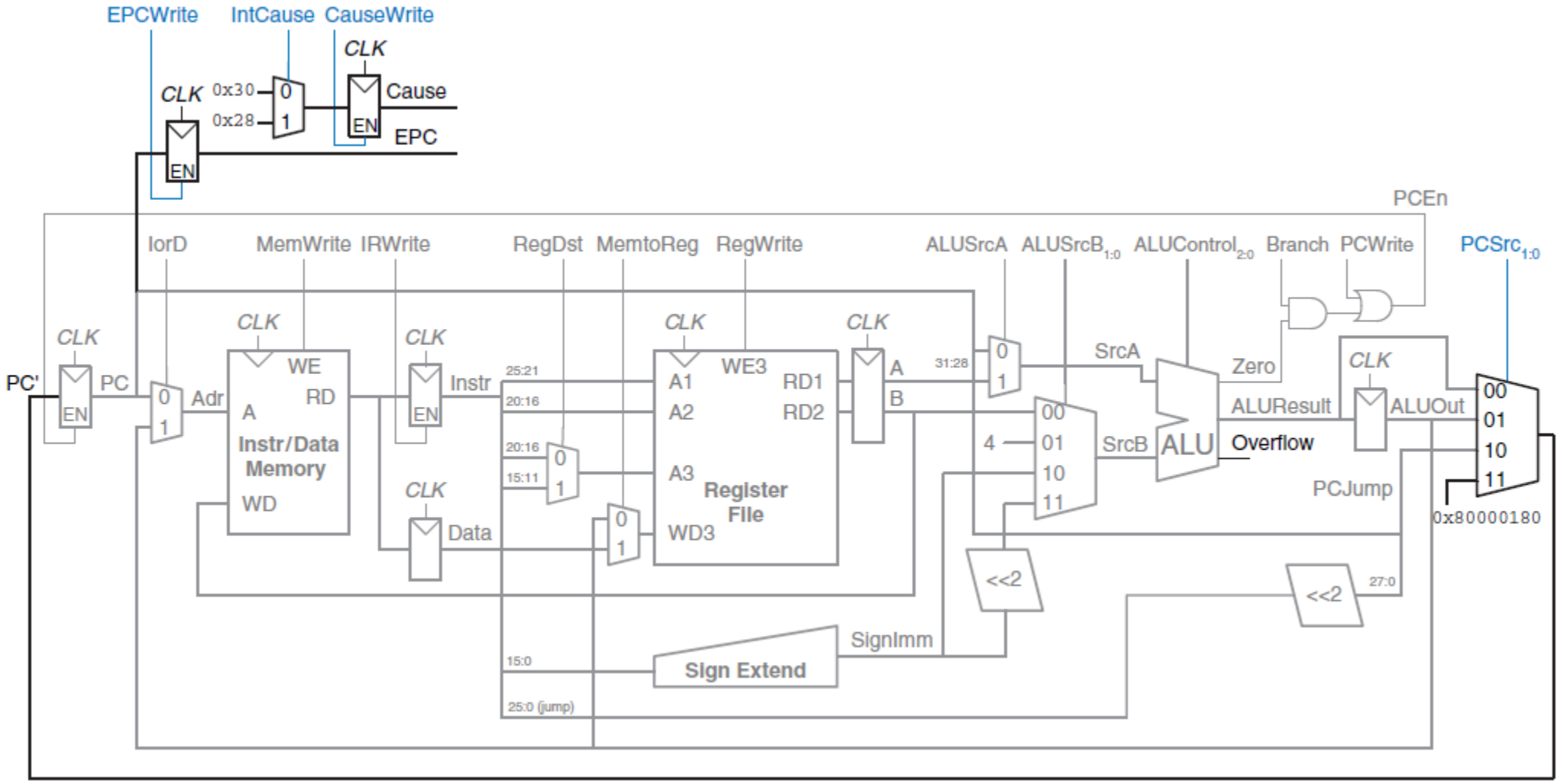
- Exceptions cause unplanned changes in the flow of a program.
- The multicycle processor will be enhanced to support two types of exceptions: undefined instructions and arithmetic overflow.
- Supporting exceptions in other microarchitectures follows similar principles.
- When an exception takes place, the processor copies the **PC** to the **EPC** register and stores a code in the **Cause** register indicating the source of the exception.
- Exception causes include 0x28 for undefined instructions and 0x30 for overflow.
- The processor then jumps to the exception handler at memory address 0x80000180.
- The exception handler is a code that is part of the OS responding to the exception.

Exception	Cause
hardware interrupt	0x00000000
system call	0x00000020
breakpoint/divide by 0	0x00000024
undefined instruction	0x00000028
arithmetic overflow	0x00000030

6. Exceptions

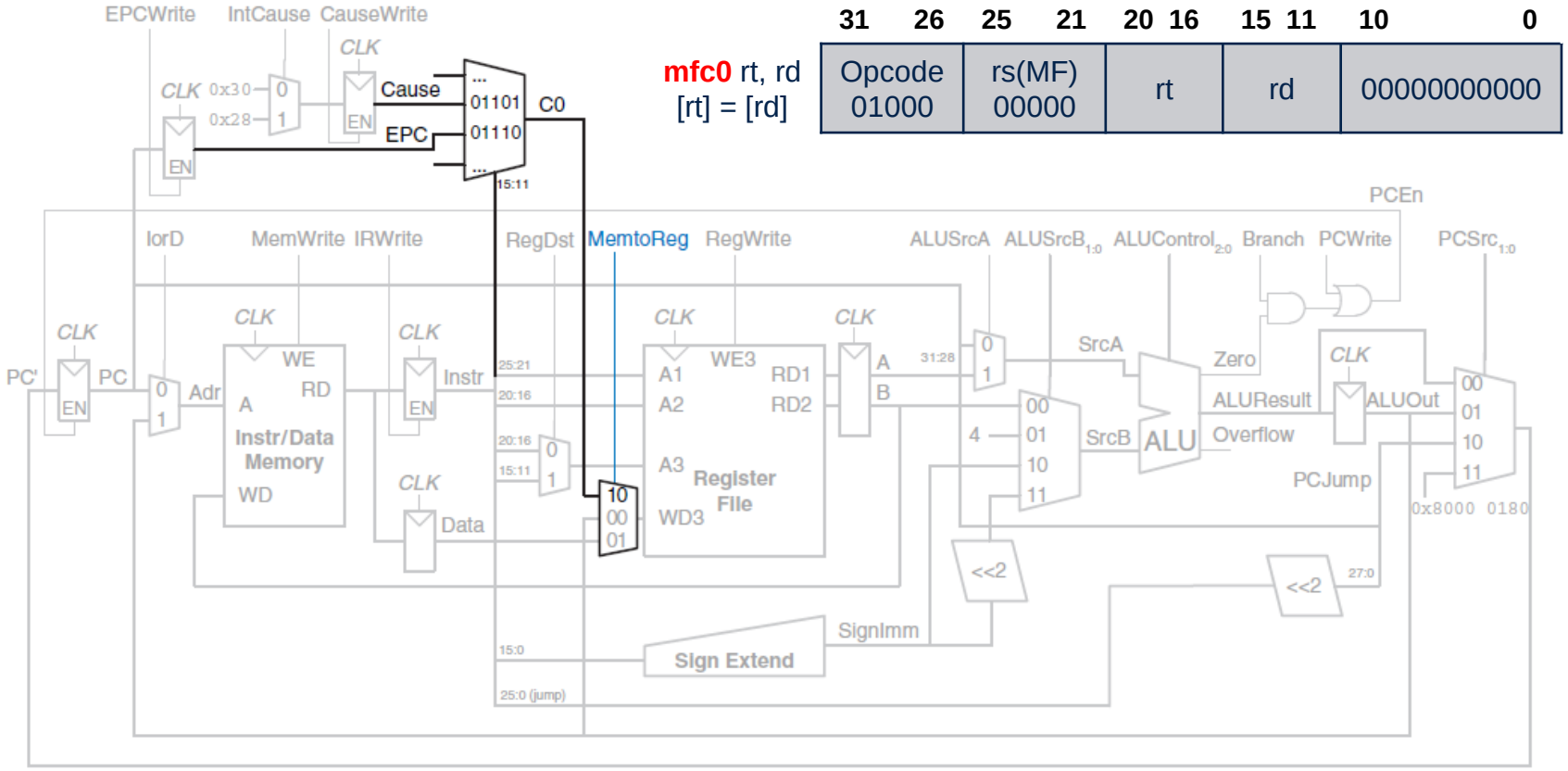
- Also, the exception registers are part of Coprocessor 0, a portion of the MIPS processor that is used for system functions.
- Coprocessor 0 defines up to 32 special-purpose registers, including **Cause** and **EPC**.
- The **Cause** register is Coprocessor 0 register 13, and **EPC** is register 14.
- The exception handler may use the mfc0 (move from coprocessor 0) instruction to copy these special-purpose registers into a general-purpose register in the register file.
- To handle exceptions, we must add **EPC** and **Cause** registers to the datapath and extend the PCSrc multiplexer to accept the exception handler address.
- The two new registers have write enables, **EPCWrite** and **CauseWrite**, to store the PC and exception cause when an exception takes place.
- The cause is multiplexed to select the appropriate code for the exception using **IntCause**.
- The ALU should also generate an overflow signal.

6. Exceptions



6. Exceptions

- To support the **mfc0** instruction, we also add a way to select the Coprocessor 0 registers and write them to the register file.
- The **mfc0** instruction specifies the Coprocessor 0 register by Instr15:11.
- In this diagram, only the **Cause** and **EPC** registers are supported.
- We add another input to the **MemtoReg** multiplexer to select the value from Coprocessor 0.
- The controller receives the overflow flag from the ALU.
- It generates three new control signals: **EPCWrite**, **CauseWrite**, and **IntCause**.

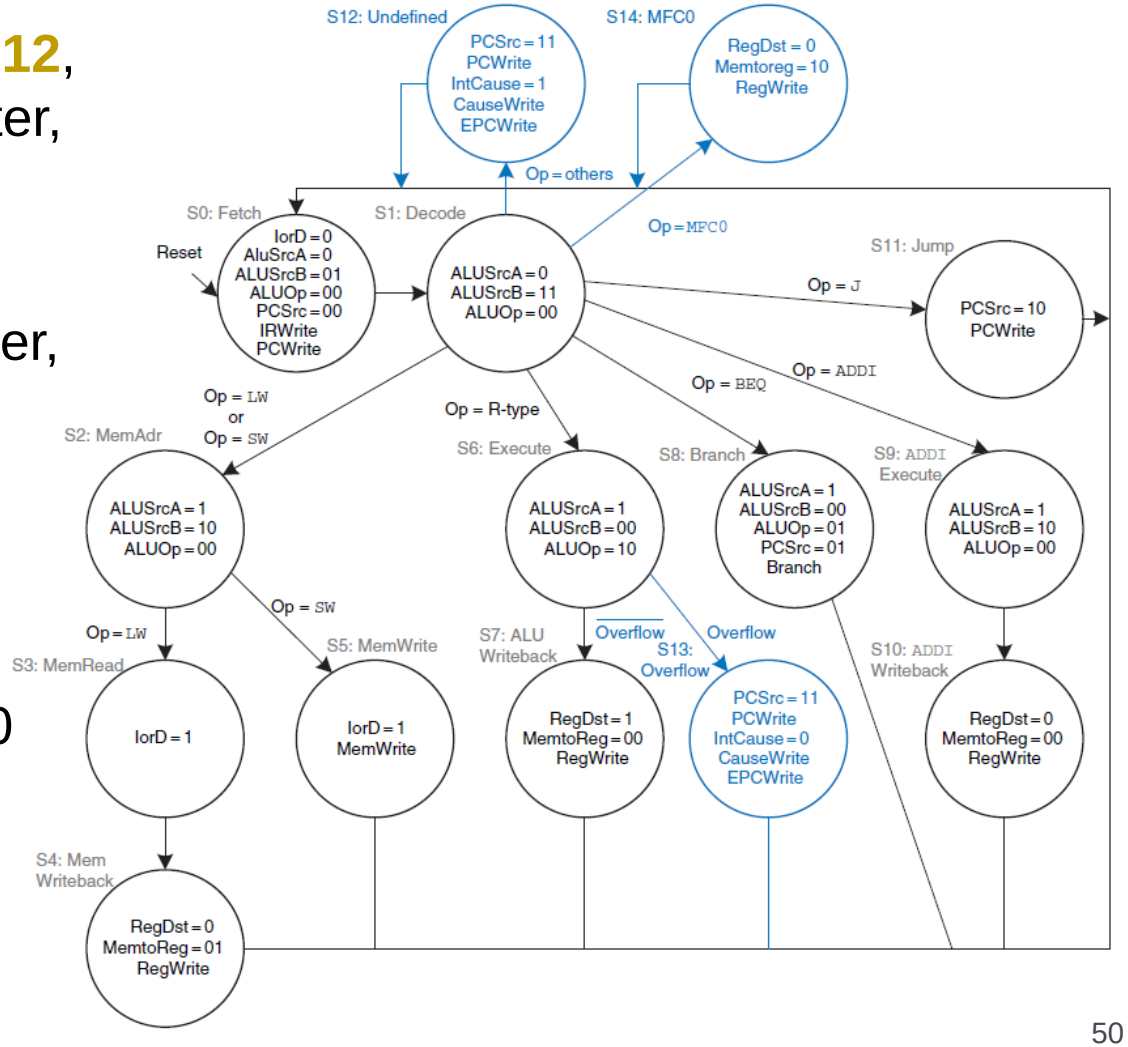


mfc0 rt, rd
[rt] = [rd]

31 26	25 21	20 16	15 11	10 0
Opcode 01000	rs(MF) 00000	rt	rd	00000000000

6. Exceptions

- The controller will include two new states to support the exceptions and another to handle **mfc0**.
- If it receives an undefined instruction, it proceeds to **S12**, saves the **PC** in **EPC**, writes 0x28 to the **Cause** register, and jumps to the exception handler.
- If it detects arithmetic overflow, it proceeds to **S13**, saves the **PC** in **EPC**, writes 0x30 in the **Cause** register, and jumps to the exception handler.
- Note: when an exception occurs, the instruction is discarded and the register file is not written.
- When a **mfc0** instruction is decoded, the processor goes to **S14** and writes the appropriate Coprocessor 0 register to the main register file.



6. Exceptions

To explain the exception handler in pipeline processor:

Example (1):

- Assume, we got a Hardware Exception (Interrupt) within cycle #3:

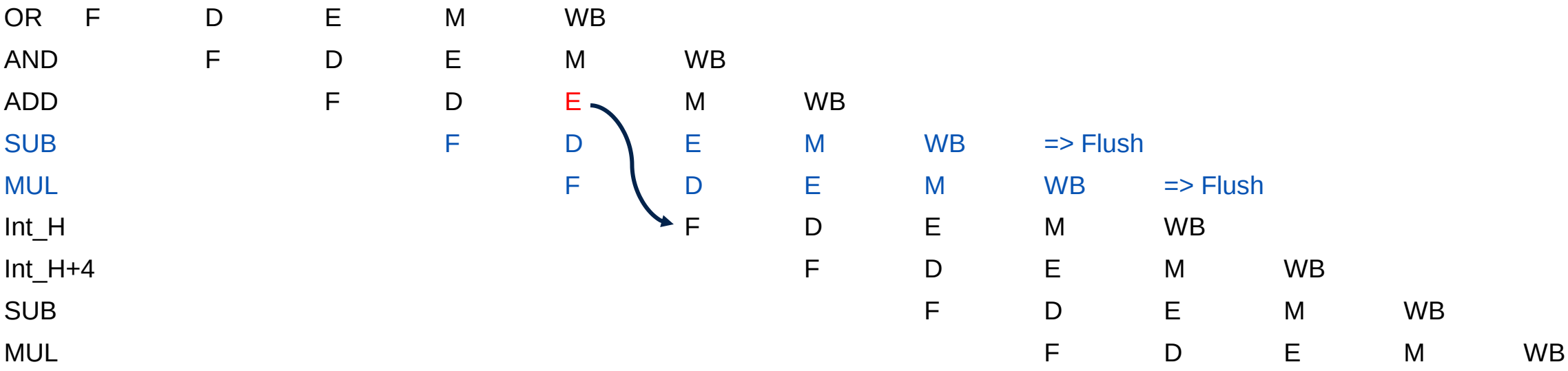
			Int						
OR	F	D	E	M	WB				
AND		F	D	E	M	WB			
ADD			F	D	E	M	WB		
Int_H				F	D	E	M	WB	
Int_H+4					F	D	E	M	WB

- It’s expected to start fetching the interrupt handler from cycles #4 as shown

6. Exceptions

Example (2):

- Assume, we got a Software Exception (Interrupt) as an overflow resulted by ADD instruction at cycle #5:



- It's expected to flush the following two instructions which are at Decode and Fetch stages, to be re-fetched again after finishing the interrupt handler.

Assignment #4

Assignment #4

- Implement the pipelined MIPS and solve all occurred hazards in pipeline.
- Implement the early branch decision hardware for branching, predicting branch is not taken.
- Do a solid testbench and apply Tests#1…6.
- Deadline: Friday, 22nd Nov 2024, at 6:00 PM.
- Deliveries: Updated Architecture (Block Diagram), Design files, testbench, simulation results, synthesis results, and a presentation for your work.
  Send a mail to: eng.abdulkareem.abotaleb@gmail.com
  Using a title of: CompArch_assignment4_<first_name>_<last_name>